

高效推理研究·零基础实操手册

给两位刚入门同学的一步步教学

2026-07-18

目录

高效推理研究·零基础实操手册	1
第 0 章: 先搞懂几个词 (5 分钟)	2
第 1 章: 把电脑环境搭好 (第 1 天)	2
第 2 章: 让模型跑起来 (第 2 天)	3
第 3 章: 两个人共用的「地基」(第 1 周一起做)	4
第 4 章·同学 A: 让模型「早点停」(第 2~4 周)	6
第 5 章·同学 B: 让算力「花在刀刃上」(第 2~4 周)	8
第 6 章: 两人如何配合 (重要)	10
第 7 章: 新手最容易踩的坑 (务必先看)	11
第 8 章: 两周内的最小目标 (照这个验收)	11
第 9 章: 遇到问题怎么办	11

高效推理研究·零基础实操手册

写给两位刚入门的同学: 从装环境到跑通实验, 一步一步来

这份手册的目标: 就算你现在完全没碰过大模型, 只要照着抄命令、抄代码, 也能在两周内跑出第一批实验结果。

你们要研究的一句话: 让 AI「想问题」的时候别想太久、别做无用功, 在保证答对的前提下省下计算。

两个人怎么分工:

- 同学 A ——让模型「早点停下来」: 模型答题时会写一长串思考过程, 很多时候后半段是废话。A 的任务是判断「什么时候可以提前收尾」。
- 同学 B ——让模型「把力气花在刀刃上」: 有一种提升正确率的采样方法 (Power Sampling) 很费算力,B 的任务是让它别在简单的地方浪费, 只在难点上多花功夫。

两人共用同一套代码和数据, 前两周一起搭好地基, 之后各做各的。

第 0 章: 先搞懂几个词 (5 分钟)

别怕术语, 先记住大白话:

术语	大白话解释
推理模型 (reasoning model)	会先写一大段「思考过程」再给答案的 AI, 比如 DeepSeek-R1。
思维链 (Chain of Thought, CoT)	就是那段思考过程, 通常很长。
token	模型眼里的「字」。一个中文字大概 1~2 个 token, 算力和钱都按 token 算。
推理效率	用「答对的比例 ÷ 花掉的 token 数」来衡量。省 token 又不掉分, 就是效率高。
checkpoint(检查点)	我们每隔一段 (比如 256 个 token) 就暂停一下, 看看模型「现在的答案是什么」。
early exit(提前停止)	发现模型已经想明白了, 就打断它, 直接让它给答案, 省下后面的 token。
logprob / entropy	模型对自己每个字有多「确定」的数值。越确定 logprob 越高、entropy 越低。这是我们判断的重要线索。
benchmark(评测集)	一堆有标准答案的题目, 用来打分。我们用 MATH、AIME(数学)、GPQA(科学)。

核心想 (整个项目就这一句):

只有当「再多想 / 再多算」还可能带来好处时, 才继续。否则就停。

第 1 章: 把电脑环境搭好 (第 1 天)

这一章两个人都要做, 各自在自己的机器 / 服务器上跑一遍。

1.1 你需要什么

- 一块 英伟达 GPU, 显存至少 24GB(比如 3090 / 4090 / A5000)。学校服务器一般有, 问导师要账号。
- Linux 系统 (服务器基本都是)。Windows 用户建议用服务器或 WSL。
- 网络能访问 Hugging Face(下载模型的地方)。国内如果慢, 后面教你用镜像。

1.2 装 Python 环境 (照抄即可)

```
# 1) 装 miniconda(如果服务器已经有 conda 就跳过)
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh      # 一路回车 + yes

# 2) 新建一个干净的环境, 叫 reason
conda create -n reason python=3.10 -y
```

```
conda activate reason
```

3) 装核心库

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install vllm transformers accelerate
pip install numpy pandas matplotlib scikit-learn tqdm
```

国内下载慢？在命令前设置镜像：

```
export HF_ENDPOINT=https://hf-mirror.com
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

1.3 验证 GPU 能用

新建文件 check.py:

```
import torch
print(" 有 GPU 吗:", torch.cuda.is_available())
print("GPU 型号:", torch.cuda.get_device_name(0))
print(" 显存 (GB):", torch.cuda.get_device_properties(0).total_memory / 1e9)
```

运行 `python check.py`, 看到 GPU 型号和显存就说明环境 OK。

第 2 章: 让模型跑起来 (第 2 天)

2.1 下载并加载一个推理模型

我们先用 DeepSeek-R1-Distill-Qwen-7B(会写长思考, 适合两个方向)。

新建 run_model.py:

```
from vllm import LLM, SamplingParams

# 第一次运行会自动下载模型 (约 15GB), 耐心等待
llm = LLM(model="deepseek-ai/DeepSeek-R1-Distill-Qwen-7B",
          dtype="bfloat16",
          gpu_memory_utilization=0.9)

# 让它做一道题
prompt = " 请解答: 12 * 15 等于多少? 一步步思考, 最后给出答案。"
params = SamplingParams(temperature=0.6, max_tokens=1024)

out = llm.generate([prompt], params)
print(out[0].outputs[0].text)
```

运行 `python run_model.py`。你会看到模型输出一大段思考 + 答案。恭喜, 第一步跑通了!

2.2 关键: 怎么拿到「模型有多确定」

我们要的不只是文字, 还要每个 token 的 logprob(确定程度)。改一下采样参数:

```

params = SamplingParams(
    temperature=0.6,
    max_tokens=1024,
    logprobs=1,          # ← 关键：返回每个生成 token 的 logprob
)

out = llm.generate([prompt], params)
o = out[0].outputs[0]

for tok_id, logprob_dict in zip(o.token_ids, o.logprobs):
    # logprob 越接近 0 越确定，越负越不确定
    lp = list(logprob_dict.values())[0].logprob
    print(tok_id, round(lp, 3))

```

entropy(不确定度) 怎么来？简单版：直接用 $-\text{logprob}$ 当作「惊讶度」，数值越大越不确定。等你们熟练了再用完整公式。两位同学都会反复用到这个信号。

第 3 章：两个人共用的「地基」（第 1 周一起做）

这一周的唯一目标：做出一个能自动答题 + 自动打分 + 自动记录的流水线。有了它，后面所有实验都是在它上面改。

3.1 准备题目数据

```

from datasets import load_dataset

# MATH500:500 道数学题，有标准答案
data = load_dataset("HuggingFaceH4/MATH-500", split="test")
print(data[0]["problem"])      # 题目
print(data[0]["answer"])       # 标准答案

```

先只取前 100 道练手，别一上来就跑 500 道浪费时间。

3.2 让模型答题并保存所有信息

对每道题，我们要存下这些东西（存成一个 .json 文件，一道题一行）：

question	题目
ground_truth	标准答案
reasoning_text	模型完整的思考过程
final_answer	模型最后给的答案
token_logprobs	每个 token 的确定程度（一个数组）
total_tokens	一共生成了多少 token
runtime	花了多少秒

参考代码 generate_dataset.py:

```

import json, time
from vllm import LLM, SamplingParams

```

```

from datasets import load_dataset

llm = LLM(model="deepseek-ai/DeepSeek-R1-Distill-Qwen-7B", dtype="bfloat16")
data = load_dataset("HuggingFaceH4/MATH-500", split="test").select(range(100))
params = SamplingParams(temperature=0.6, max_tokens=4096, logprobs=1)

with open("results.jsonl", "w") as f:
    for item in data:
        prompt = f" 请解答下面的数学题，一步步思考，最后用 \\boxed{{}} 给出答案。\\n题目:{item['problem']}"
        t0 = time.time()
        out = llm.generate([prompt], params)[0].outputs[0]
        logprobs = [list(d.values())[0].logprob for d in out.logprobs]
        record = {
            "question": item["problem"],
            "ground_truth": item["answer"],
            "reasoning_text": out.text,
            "token_logprobs": logprobs,
            "total_tokens": len(out.token_ids),
            "runtime": round(time.time() - t0, 2),
        }
        f.write(json.dumps(record, ensure_ascii=False) + "\\n")
    print("done:", item["problem"][:30], "tokens=", len(out.token_ids))

```

3.3 自动判分 (把答案抠出来对比)

数学答案格式五花八门 (分数、小数、`\boxed{}`)，需要清洗后再比。先用简单版：

```

import re

def extract_answer(text):
    """ 从模型输出里抠出 \\boxed{} 里的答案 """
    m = re.findall(r"\\boxed\\{([\\^}]*\\)", text)
    return m[-1].strip() if m else None

def is_correct(pred, gold):
    if pred is None:
        return False
    # 去掉空格，统一比较 (先粗糙比，后面再优化)
    return pred.replace(" ", "") == str(gold).replace(" ", "")

```

注意：一开始判分会会有很多「其实答对了但格式不同」被误判。这很正常。第 2 周专门花时间优化判分 (处理分数、约分、等价表达)。这是脏活但很重要，判分不准后面所有结论都不可信。

3.4 分工小提示

第 1 周末尾，两个人在同一套流水线上各加一块：

- 同学 A 补上「检查点」功能 (见第 4 章)：每隔 256 token 暂停，记录当前答案。
- 同学 B 补上「重新采样」功能 (见第 6 章)：能从某个位置往后重新生成，并算这段文字的 likelihood。

第 4 章·同学 A: 让模型「早点停」(第 2~4 周)

4.1 一句话任务

模型思考很长,但常常前半半就已经想出正确答案了,后半半在反复验证、重复啰嗦。A 要做的:判断「什么时候可以安全地打断它」。

4.2 第一步:做「检查点」,看答案怎么变化(第 2 周)

在模型思考过程中,每隔 256 个 token 就暂停一次,问它:「如果现在就让你回答,你的答案是什么?」

做法:把「当前已经想到的部分」+ 一句「所以最终答案是」拼起来,让模型补一个短答案:

```
def peek_answer(llm, question, reasoning_prefix):
    """ 在思考中途偷看一眼当前答案 """
    prompt = (f" 题目:{question}\n"
              f" 思考过程:{reasoning_prefix}\n"
              f"</think>\n所以最终答案是 \boxed{{}}")
    params = SamplingParams(temperature=0, max_tokens=20)
    out = llm.generate([prompt], params)[0].outputs[0]
    return extract_answer("\boxed{{}} " + out.text)
```

对一道题,每个检查点都偷看一次,你会得到一串答案,比如:

```
第 256 token 时: 答案 = 42    (错)
第 512 token 时: 答案 = 17    (对!)
第 768 token 时: 答案 = 17    (对, 没变)
第 1024 token 时: 答案 = 17   (对, 还在重复验证)
... 模型又写了 2000 个 token 才停...
```

这张「答案变化表」就是你研究的金矿。它告诉你:这道题其实第 512 token 就该停了,后面 2000 多 token 全浪费了。

4.3 第二步:算出「理论上最多能省多少」(第 2 周)

对每道题,找到最早那个答案已经正确、而且之后再没变错的检查点,叫它 oracle 停止点。

统计所有题的 oracle 停止点,你能得到一句很有说服力的话:

「如果有个完美的裁判,这个模型平均能省掉 40% 的思考 token,而正确率完全不变。」

这就是你方法的上限,也是你要努力逼近的目标。

4.4 第三步:训练一个「小裁判」来预测停止时机(第 3 周)

我们没有完美裁判,得自己训练一个轻量小模型(注意:不动大模型,只训练一个几十行的小分类器)。

思路:在每个检查点,收集一堆「线索」,让小模型预测「现在停,安不安全?」

收集的线索(特征):

```
def make_features(logprobs_recent, answer_history):
    return {
        "avg_logprob": sum(logprobs_recent) / len(logprobs_recent), # 最近这段有多确定
        "min_logprob": min(logprobs_recent),                        # 最不确定的那个字
```

```

        "answer_same_count": count_consecutive_same(answer_history), # 答案连续没变几次了
        "answer_changed": answer_history[-1] != answer_history[-2], # 刚刚答案变了吗
        "reasoning_length": len(logprobs_recent), # 已经想了多长
    }

```

训练标签 (答案): 用 4.2 得到的数据, 标注每个检查点「在这里停是对是错」:

- 如果这里的答案 = 标准答案, 且和完整思考的最终答案一致 → 标签「可以停」(记为 1)
- 如果这里答错、但继续想会答对 → 标签「不能停」(记为 0)

用最简单的模型开始:

```

from sklearn.linear_model import LogisticRegression
import numpy as np

X = np.array(all_features) # 所有检查点的特征
y = np.array(all_labels) # 0= 不能停, 1= 可以停
model = LogisticRegression().fit(X, y)

# 预测: 某个检查点该不该停
prob_can_stop = model.predict_proba(new_feature)[0][1]
if prob_can_stop > 0.9:
    print(" 可以停了!")

```

先用 LogisticRegression(逻辑回归, 最简单), 跑通后再换 MLP 或加更多特征。

4.5 第四步: 装上「刹车」跑完整流程 (第 3~4 周)

把小裁判接进生成流程:

```

每写 256 个 token → 偷看当前答案 → 算特征 → 小裁判打分
    如果「可以停」的概率 > 阈值, 并且答案已经连续几次没变:
        → 让模型立刻给最终答案, 停!
    否则:
        → 继续写

```

4.6 最重要的安全指标: 别停错了

最怕的错误是: 本来再想想就能答对, 结果你提前打断, 答错了。这个必须严格控制。

在一个专门的「校准集」上调阈值, 让「停错的比例」低于比如 5%。宁可少省一点 token, 也不能明显掉正确率。

4.7 要对比的基线方法 (证明你的方法更好)

把你的方法和这些简单方法比:

1. 不停, 写到底 (原始);
2. 固定写 1024 / 2048 token 就停;
3. 只看 entropy, 低于某值就停;
4. 只看「答案连续 3 次没变」就停;
5. oracle(完美裁判, 上限)。

4.8 最终要画的一张关键图

- 横轴: 平均花了多少 token(算力)
- 纵轴: 正确率
- 每种方法画一条线。你的线如果在别人上方 (同样 token 下正确率更高, 或同样正确率下 token 更少), 就赢了。

4.9 A 的六周节奏表

周	做什么
1	和 B 一起搭流水线; 加检查点功能; 画出答案变化表
2	优化判分; 算 oracle 上限; 做好特征和标签
3	训练逻辑回归小裁判; 接入生成流程跑通
4	调阈值控制停错率; 加更多特征; 换 MLP
5	上全量 MATH500 + AIME + GPQA, 多个模型
6	画 Pareto 图; 分析失败案例; 写报告

第 5 章·同学 B: 让算力「花在刀刃上」(第 2~4 周)

5.1 一句话任务

有个方法叫 Power Sampling, 能提高正确率, 但很费算力: 它会把生成的文字反复「重新采样」很多遍。B 要做的: 让它别傻乎乎每个地方都重采一样多次, 而是简单的地方少采、难点多采。

5.2 先搞懂 Power Sampling 在干嘛 (用大白话)

普通生成: 模型一次性写完答案。

Power Sampling: 模型写完后, 反复挑一段重写, 如果重写的版本「模型觉得更顺 (likelihood 更高)」就采纳, 否则丢弃。重复很多轮, 答案质量会变好。这个「重写—比较—采纳/丢弃」的机制叫 MCMC。

问题在于:

- 它对每一段都重写固定次数 (比如 8 次), 不管这段是不是已经很好了;
- 挑哪段重写是随机的, 可能老在无关紧要的地方折腾;
- 很多重写的版本被丢弃了, 但生成它们花的算力已经没了。

5.3 第一步: 先把固定版本跑通 (第 2 周)

用 Qwen2.5-Math-7B(专门做数学, 适合这个方法)。目标不是一步到位复现论文所有数字, 而是: 跑通流程、看懂开销在哪。

你需要实现两个基本操作:

(1) 算一段文字的 likelihood(模型觉得它有多「顺」):

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-Math-7B")
```

```

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-Math-7B",
                                              torch_dtype=torch.bfloat16, device_map="cuda")

def seq_loglik(text):
    ids = tok(text, return_tensors="pt").input_ids.cuda()
    with torch.no_grad():
        logits = model(ids).logits
        logp = torch.log_softmax(logits[:, :-1], dim=-1)
        picked = logp.gather(2, ids[:, 1:].unsqueeze(-1)).squeeze(-1)
    return picked.sum().item()      # 越大表示模型越觉得这段顺

```

(2) 从某个位置往后重新生成 (重采样一段后缀):

```

def resample_suffix(prefix_text, max_new=128):
    ids = tok(prefix_text, return_tensors="pt").input_ids.cuda()
    out = model.generate(ids, max_new_tokens=max_new, do_sample=True, temperature=0.8)
    return tok.decode(out[0][ids.shape[1]:])

```

接受/拒绝规则 (MCMC 的核心): 重写后如果 likelihood 更高就接受, 否则按一个概率决定。先照最简单的规则实现, 跑通再说。

这一周一定要记录的开销数据:

```

每道题重写了多少次
一共生成了多少 token (包括被丢弃的)
被丢弃的 token 有多少      ← 这就是浪费, 你要削减的目标
接受率是多少
正确率

```

5.4 第二步: 让它「该停就停」(第 3 周)

观察一个现象: 某段文字重写几次后, likelihood 就不怎么涨了 (到顶了), 再重写就是浪费。

规则: 如果连续几次重写, likelihood 几乎没提升 (或者答案一直没变), 就停止重写这一段, 进入下一段。

```

def should_stop_mcmc(loglik_history, min_steps=2):
    if len(loglik_history) < min_steps + 1:
        return False
    recent_gain = loglik_history[-1] - loglik_history[-2]
    return abs(recent_gain) < 0.01      # 几乎不涨了 → 停

```

这样, 好写的段落一两次就过, 难的段落才多花功夫。对比固定 8 次, 能省很多。

5.5 第三步: 让它「挑难点重写」(第 4 周)

原方法随机挑位置重写。你改成: 优先挑那些「模型没把握」的位置。

怎么找难点? 给每个位置打个「关键分」:

```

def criticality_score(logprobs, position):
    # entropy 高 (不确定)、logprob 突然掉下来的地方, 更可能是难点/转折点
    return -logprobs[position]      # 简单版: 越不确定分越高

```

然后优先在高分位置重写, 而不是随机挑。

重要的坑 (数学正确性): 一旦你不再「随机均匀挑位置」,MCMC 的接受概率公式要相应修改, 否则方法在数学上就不对了。这部分建议先在一个「玩具例子」(很小的词表、很短的序列) 上验证: 能精确算出理论分布, 再看你采样出来的频率对不对。这个验证做扎实了再上真模型。导师会帮你过这部分公式。

5.6 要对比的基线方法

- 1. 普通生成 (不重采样);
- 2. 原始固定次数 Power Sampling;
- 3. 你的「自适应停止」版;
- 4. 你的「挑难点」版;
- 5. 两个都用的完整版。

5.7 最终要画的图

- 图 1: 横轴 = 总共花的 token, 纵轴 = 正确率。你的方法应该用更少 token 达到接近的正确率。
- 图 2: 展示重写次数在不同位置的分布——证明「难点多花、简单少花」确实发生了。

5.8 B 的六周节奏表

周	做什么
1	和 A 一起搭流水线; 实现「算 likelihood」和「重采样后缀」两个操作
2	跑通固定版 Power Sampling; 记录开销和浪费的 token
3	加「自适应停止」; 和固定版对比省了多少
4	加「挑难点重写」; 在玩具例子上验证公式正确
5	上 MATH500 等完整实验, 测正确率和延迟
6	画 Pareto 图; 分析; 写报告

第 6 章: 两人如何配合 (重要)

- 第 1 周: 并肩坐一起, 把第 3 章的流水线搭好。这是两个人共同的地基, 值得认真做。判分、数据格式一定要统一, 否则后面结果没法比。
- 第 2 周起: 各做各的, 但共用一个代码仓库 (用 Git)。约定好文件夹:

generation/	生成相关代码 (共用)
evaluation/	判分代码 (共用)
early_exit/	同学 A 的代码
sampling/	同学 B 的代码
results/	实验结果 (不进 Git, 太大)
figures/	画的图

- 每周碰一次头, 互相讲一下进展和卡住的地方。很多坑是共通的。
- 两个人最后是两篇独立的报告 / 论文, 但共享同一套基础设施。

第 7 章: 新手最容易踩的坑 (务必先看)

1. 一上来就跑 500 道题。→ 先跑 5 道调试, 再跑 100 道, 最后才 500 道。省时间省显存。
 2. 判分不准还不知道。→ 一定要人工检查几十个判分结果, 确认「答对被判对、答错被判错」。判分错, 一切白搭。
 3. 显存爆了 (OOM)。→ 调小 max_tokens, 调小 gpu_memory_utilization, 或一次只处理一道题。
 4. 只报告「省了多少 token」。→ 还要报告实际跑了多少秒 (wall-clock latency)。有时候 token 省了但时间没省。
 5. A 停太早导致答错、B 改公式导致方法失效。→ 这两个是各自方向最危险的错误, 专门留时间控制。
 6. 不用 Git, 代码丢了 / 覆盖了。→ 第一天就建仓库, 每天 commit。
 7. 实验结果不记录参数。→ 每次实验把「用了哪个模型、什么温度、什么阈值」都存进结果文件, 否则事后不知道这个数是怎么来的。
-

第 8 章: 两周内的最小目标 (照这个验收)

如果你们照做, 两周后应该能拿出:

共同 (第 1 周末): - [] 环境搭好, 模型能跑 - [] 100 道 MATH 题跑完, 结果存进 results.jsonl - [] 判分能用, 能算出模型原始正确率

同学 A (第 2 周末): - [] 检查点功能能用, 画出至少 5 道题的「答案变化表」- [] 算出 oracle 上限: 「理论上能省 X% token」

同学 B (第 2 周末): - [] 固定版 Power Sampling 跑通 - [] 记录出「浪费在被丢弃 token 上的算力有多少」

拿到这些, 就说明地基稳了, 后面是水到渠成的迭代。

第 9 章: 遇到问题怎么办

- 代码报错 → 先把报错最后几行复制, 搜索或问导师/助教。90% 的错别人都遇到过。
 - 概念不懂 → 回到第 0 章的大白话表, 或者问「这个东西到底想解决什么问题」。
 - 不知道下一步 → 看自己的六周节奏表, 做当周那一格就行, 别想太远。
 - 结果不理想 → 正常。研究就是不断试错。先保证流程对, 再优化效果。
-

一句话总结

同学 A: 教 AI 「想明白了就别啰嗦」。同学 B: 教 AI 「把力气花在难题上, 别在简单题上瞎折腾」。

两个人做的是同一件事的两面: 只在值得的时候, 才多花算力。

加油, 一步一步来, 你们可以的。